ASSIGNMENT 4
MACHINE LEARNING

ADITRI JAIN (2024027)

Random seed : 2024027

Activation Functions and Their Derivatives (P1)

a)Implementation and numerical stability

We implemented sigmoid, tanh, ReLU, leaky ReLU and softplus, along with their derivatives. Each function works with NumPy arrays and preserves the input shape.

To avoid numerical overflow, we used stable expressions for sigmoid and softplus. All five functions passed the stability tests for input values up to ±1000.

Numerical derivative verification

We used the central difference formula: (with $\epsilon = 10^{-6}$)

$$f'(z) \approx \frac{f(z+\epsilon) - f(z-\epsilon)}{2\epsilon}$$

Activation	Maximum absolute error
Sigmoid	4.8×10^{-11}
Tanh	1.2×10^{-11}
ReLU	1.4×10^{-10}
Leaky ReLU	1.4×10^{-10}
Softplus	8.9×10^{-11}

All errors were below the required threshold of 10^{-6}, confirming that the implemented derivatives agree with numerical differentiation at the tested points.

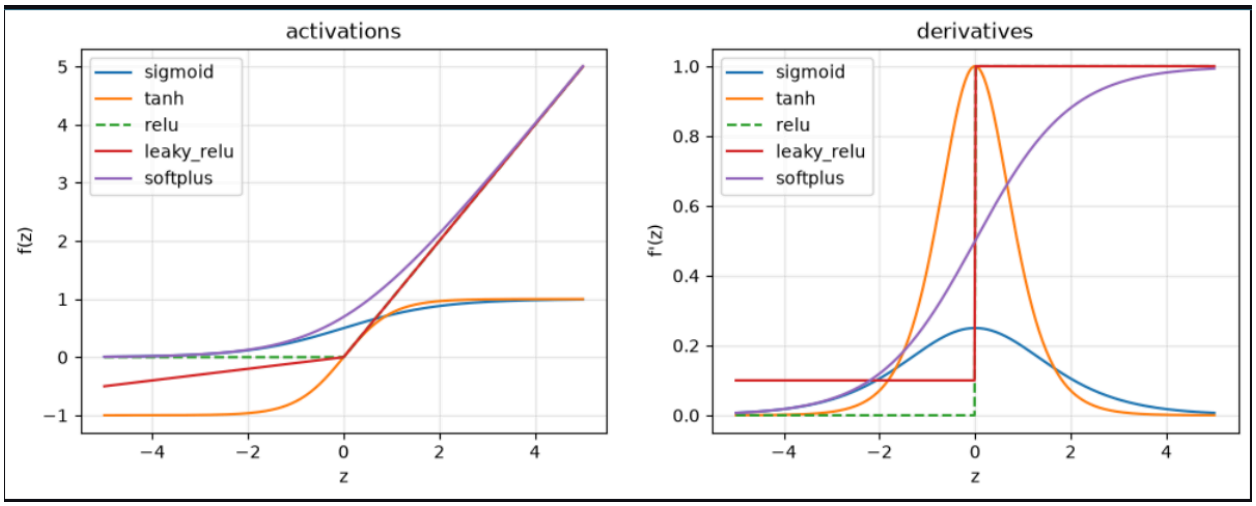


Figure 1. Activation functions (left) and their derivatives (right). The curves show how the functions and their slopes change with input.

Maximum derivative values

Function	Maximum on grid	f′(−2)	f′(0)	f′(2)
Sigmoid	0.2500	0.1050	0.2500	0.1050
Tanh	0.9998	0.0707	1.0000	0.0707
ReLU	1.0000	0.0000	0.0000	1.0000
Leaky ReLU	1.0000	0.1000	0.1000	1.0000
Softplus	0.9933	0.1192	0.5000	0.8808

The grid does not contain zero exactly, which explains why the observed maximum derivative of tanh is slightly below 1. Softplus approaches a derivative of 1 as the input increases.

Verifying the tanh–sigmoid identity

We verified:

$$\tanh(z) = 2\sigma(2z) - 1$$

The maximum absolute difference was 3.3×10^{-16}, which is effectively zero within floating-point precision.

Checkpoint answers

(i) The derivative values at −2, 0 and 2 are given in the table above.

(ii) Sigmoid has a maximum derivative of 0.25. During backpropagation, multiplying many small derivatives can make gradients extremely small, so early layers learn slowly. This is called the vanishing gradient problem.

(iii) Differentiating the identity gives:

$$\tanh'(z) = 4\sigma'(2z)$$

Therefore, the maximum derivative of tanh is 1, which is four times the maximum derivative of sigmoid.

3. Softmax and Cross-Entropy (P3)

Implementation

We implemented softmax, mean cross-entropy loss and the gradient of the loss with respect to the logits.

For numerical stability, we subtracted the maximum value of each row before applying the exponential function.

$$p_{ij} = \frac{e^{Z_{ij} - m_i}}{\sum_k e^{Z_{ik} - m_i}}, \quad m_i = \max_j Z_{ij}$$

The softmax output has the same shape as the input logits, and every row sums to 1.

Gradient verification

For a batch of n examples:

$$\frac{\partial L}{\partial Z} = \frac{P - Y}{n}$$

We checked this gradient using central differences on a random 5 × 4 logit matrix.

Test	Result
Softmax shape and row sums	PASS
Gradient check maximum error	1.8×10^{-10}

Adding a constant to logits	Maximum difference 1.2×10^{-15}
Naive softmax on large logits	NaN
Stable softmax on large logits	Valid probabilities

For logits [1000, 1001, 1002], the stable implementation returned:

$$[0.0900,\ 0.2447,\ 0.6652]$$

Checkpoint

For logits [2.0, 1.0, 0.1] and label 1:

Quantity	**Result**
Softmax probabilities	[0.6590, 0.2424, 0.0986]
Cross-entropy loss	1.4170
Gradient	[0.6590, −0.7576, 0.0986]
Gradient row sum	0.0000

Concept checks

(i) Subtracting the same constant from every logit does not change softmax because the exponential factor cancels between the numerator and denominator. It also prevents overflow for large logits.

(ii) Each gradient row sums to zero because the probabilities sum to 1 and the one-hot labels also sum to 1. This is consistent with softmax being unchanged when all logits are shifted by the same amount.

4. Data Pipelines: CIFAR-10 and Wine (P4)

Data preparation

For CIFAR-10, we randomly selected 10,000 training images using the assigned seed. We used 8,000 images for training, 2,000 for validation, and the first 2,000 official test images for testing.

We converted the pixel values to the range [0, 1], calculated the mean and standard deviation using only the training set, and applied the same normalization to all three sets.

Each image was flattened into a 3,072-dimensional vector in channel-first order.

Dataset shapes

Dataset	Images	Flattened shape
Training	8,000	(8000, 3072)
Validation	2,000	(2000, 3072)
Test	2,000	(2000, 3072)

Channel statistics

Set	Mean (R, G, B)	Std (R, G, B)
Train before	0.4910, 0.4816, 0.4458	0.2473, 0.2439, 0.2617
Train after	0.0000, 0.0000, 0.0000	1.0000, 1.0000, 1.0000

Val after	−0.0038, −0.0116, −0.0088	0.9959, 0.9938, 0.9918
Test after	0.0257, 0.0293, 0.0369	0.9993, 0.9939, 1.0034

The training set has approximately zero mean and unit standard deviation after normalization. The validation and test statistics are slightly different because their images were normalized using the training statistics.



Figure 2. One example from each of the ten CIFAR-10 classes.

Training images per class

Class	Count	Class	Count
Airplane	820	Dog	790
Automobile	797	Frog	802
Bird	769	Horse	798

Cat	822	Ship	786
Deer	829	Truck	787

Total: 8,000 images.

For the Wine dataset used in later problems, we used a seeded 70%/15%/15% train-validation-test split, giving 124, 27 and 27 samples respectively. This split was our implementation choice.

Concept checks

(i) Validation and test data must not be used to calculate normalization statistics because this would leak information from unseen data into training. Using only training statistics gives a fairer evaluation.

(ii) Flattening removes the explicit spatial arrangement of pixels. As a result, an MLP cannot directly exploit nearby pixel patterns as a convolutional network can, making image recognition harder.

5. Forward Pass Through an MLP (P5)

Network architecture

We implemented a multilayer perceptron for Wine classification using NumPy.

13 input features → 16 hidden neurons → 8 hidden neurons → 3 output logits

The hidden layers use ReLU, and the output layer produces logits that are passed to softmax. We initialized weights using He initialization and set biases to zero.

Parameter count

$$(13 \times 16 + 16) + (16 \times 8 + 8) + (8 \times 3 + 3) = \boxed{387}$$

The formula and direct count of NumPy array entries both gave 387 parameters.

Forward-pass shapes

Layer	Pre-activation Z	Activation A
Input	—	(5, 13)
Hidden 1	(5, 16)	(5, 16)
Hidden 2	(5, 8)	(5, 8)
Output	(5, 3)	(5, 3)

Initial loss comparison

Experiment	Loss
Expected uniform prediction, ln(3)	1.0986
Original initialization	2.6385
Last-layer weights multiplied by 0.01	1.1085

The initial loss is higher than ln(3) because the randomly initialized network can make confident but incorrect predictions. After scaling down the final-layer weights, the logits become close to zero and the predicted probabilities become nearly uniform, bringing the loss close to ln(3).

Vectorization check

We compared the output from processing five samples together with processing them one by one.Maximum absolute difference: 6.7×10^{-16}.

The results matched within floating-point precision, confirming that the batch implementation is consistent with the sample-wise implementation.

Hidden activation statistics

Activation	Mean	Standard deviation
ReLU	0.5881	0.8993
Tanh	0.0005	0.7241
Sigmoid	0.4987	0.2664

Tanh produces activations centered close to zero, while sigmoid produces mostly positive activations. ReLU has a positive mean because negative inputs become zero.

Concept checks

(i) We keep the output layer linear because softmax is applied to its logits when calculating the loss. Applying a hidden activation at the output would unnecessarily restrict the logits.

(ii) The initial loss is close to ln(k) when the model predicts approximately equal probabilities for all k classes.

(iii) Sigmoid activations are not zero-centered, which can make optimization less efficient. Sigmoid can also saturate, producing small gradients and slowing learning.

6. Backward Pass and Gradient Checking (P6)

Backpropagation implementation

We implemented backpropagation for the MLP using NumPy, including the L2 regularization term.

The loss is:

$$J = -\frac{1}{B}\sum_{i=1}^{B}\log P_{i,y_i} + \frac{\lambda}{2}\sum_{l}\|W_l\|_F^2$$

The main gradient equations are:

$$dZ_L = \frac{P - Y}{B}$$

$$dW_l = A_{l-1}^T dZ_l + \lambda W_l$$

$$db_l = \sum_i (dZ_l)_{i,:}$$

$$dZ_{l-1} = (dZ_l W_l^T) \odot \phi'(Z_{l-1})$$

We stored the intermediate values during the forward pass and reused them to calculate gradients for every layer.

Gradient verification

We compared the analytical gradients with numerical gradients obtained using central differences.

The worst relative error for the correct implementation was:

$$\boxed{8.97 \times 10^{-7}}$$

This small error supports the correctness of the backward-pass implementation.

Bug detection experiment

We intentionally introduced three common mistakes, one at a time, and measured their effect on gradient checking.

Implementation	Worst relative error
Correct backpropagation	8.97×10^{-7}
Bug 1: Missing division by batch size	1.0000
Bug 2: Using Z instead of A	1.0000
Bug 3: Missing activation derivative	1.0000

Bug 1: Without dividing by the batch size, the output gradient is incorrectly scaled. This error propagates through the network and affects all parameter gradients.

Bug 2: The weight-gradient formula requires the previous layer's activation, not its pre-activation. This mistake affected the gradients of W2 and W3 in our experiment.

Bug 3: Omitting the activation derivative breaks the chain rule when propagating gradients to earlier layers. The final layer remains correct, but earlier-layer gradients become incorrect.

All three bugs were detected by the gradient check.

Single-batch overfitting test

We trained a smaller network with architecture $13 \to 8 \to 3$ on the first eight Wine training samples.

Metric	Result
Learning rate	0.5
Initial loss	1.2289

Final loss	0.0095
Steps required	24
Maximum allowed steps	500

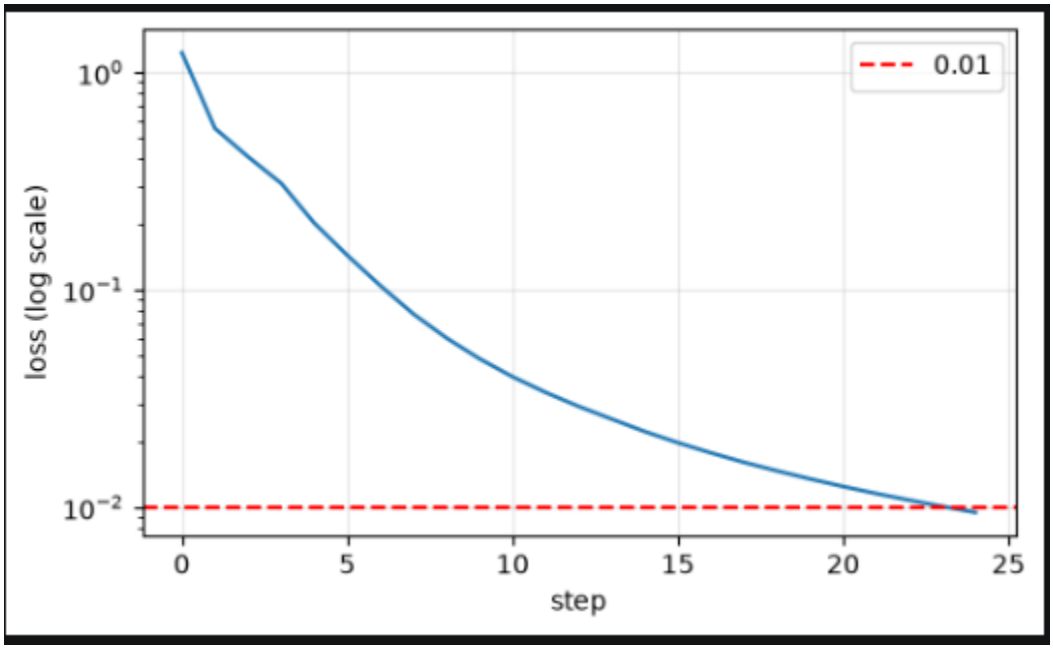


Figure 3. Training loss decreases below 0.01 after 24 gradient descent steps.

The network successfully learned the small batch. This is a useful test because it shows that the gradients and parameter updates work together during training.

Concept checks

(i) Central differences use information from both sides of a point and generally give a more accurate derivative approximation than one-sided differences for the same small step size.

(ii) A model with incorrect gradients may still show decreasing loss because the updates can sometimes point in a useful direction. However, training may be slower, unstable or converge to a worse result.

(iii) Gradient checking verifies the correctness of individual derivatives. The overfitting test checks whether the complete training process, including forward propagation, backward propagation and parameter updates, can successfully learn a small dataset.

7. Training an MLP on CIFAR-10 (P7)

Network architecture and setup

We trained an MLP from scratch using NumPy with the architecture:

3072 → 256 → 128 → 10

We used ReLU activation, He initialization, mini-batches of 128, momentum of 0.9 and float32 arrays.

Parameter count and initial loss

Metric	Value
Total parameters	820,874
First-layer parameters	786,688
Percentage in first layer	95.84%
Initial loss	2.8329
ln(10)	2.3026

Most parameters belong to the first layer because every flattened image has 3,072 input values.

The initial loss is higher than ln(10), indicating that the random network does not initially produce uniform predictions.

Single-batch overfitting test

We trained the network on 64 images with learning rate 0.05 and no regularization.

Metric	Result
Initial loss	2.8611
Final loss	0.0476
Steps required	21
Target loss	Below 0.05

The network reached the target in 21 steps, showing that it has enough capacity to memorize a small batch.

Training for 15 epochs

We trained the model with learning rate 0.01, momentum 0.9 and no L2 regularization.

Metric	Result
Best validation epoch	15
Training accuracy at epoch 15	96.99%
Validation accuracy	43.30%

Test accuracy	43.15%
Training loss at epoch 15	0.1392
Validation loss at epoch 15	2.5727

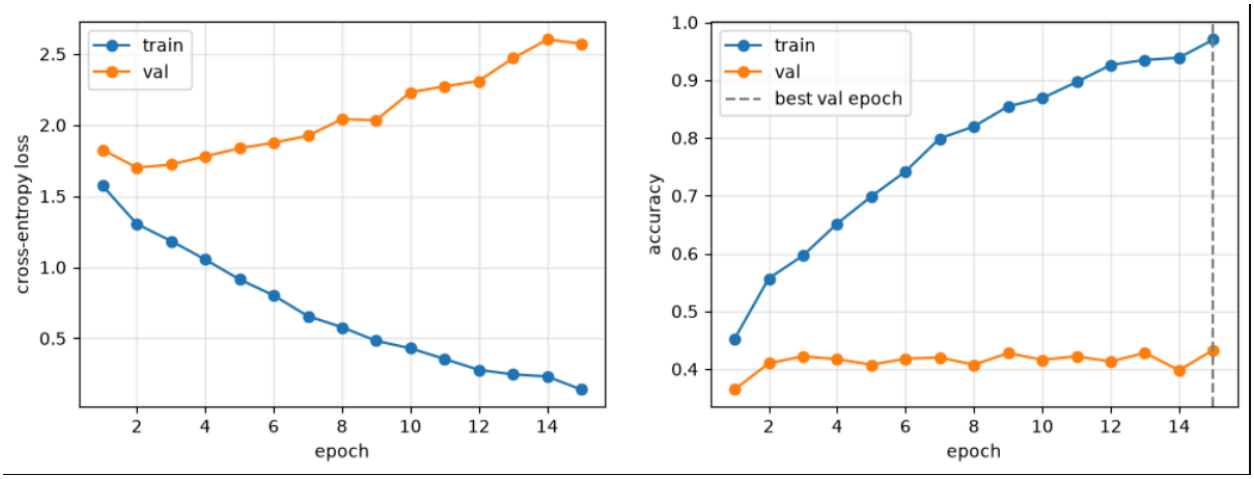


Figure 4. Training and validation loss and accuracy over 15 epochs.

The model learned the training images well, but its validation accuracy remained much lower. The validation loss also increased in later epochs, suggesting that the network was becoming overconfident on images it could not classify correctly.This is a clear sign of overfitting.

Visualizing first-layer weights

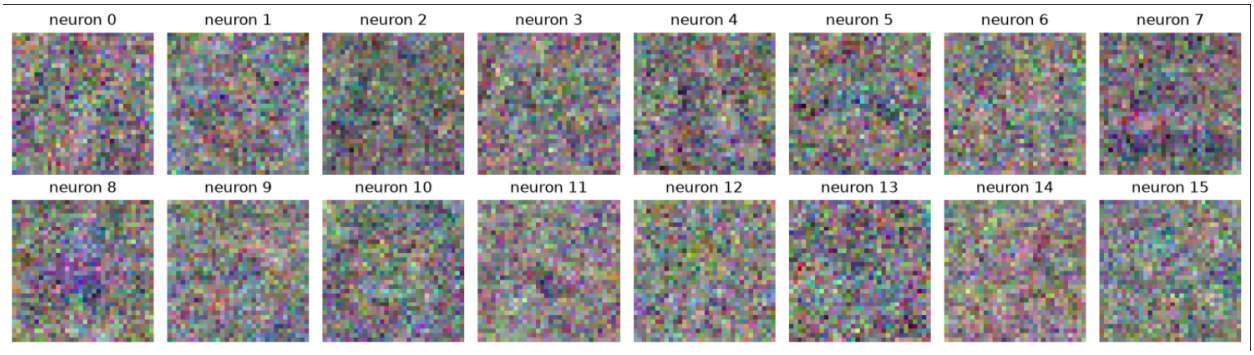


Figure 5. First-layer weights reshaped into 3 × 32 × 32 images, individually rescaled to [0, 1].

The weight images help us understand which pixel patterns the first layer responds to. They may show color differences, broad spatial patterns and other learned features, but they are not necessarily recognizable objects.

Confusion matrix and per-class accuracy

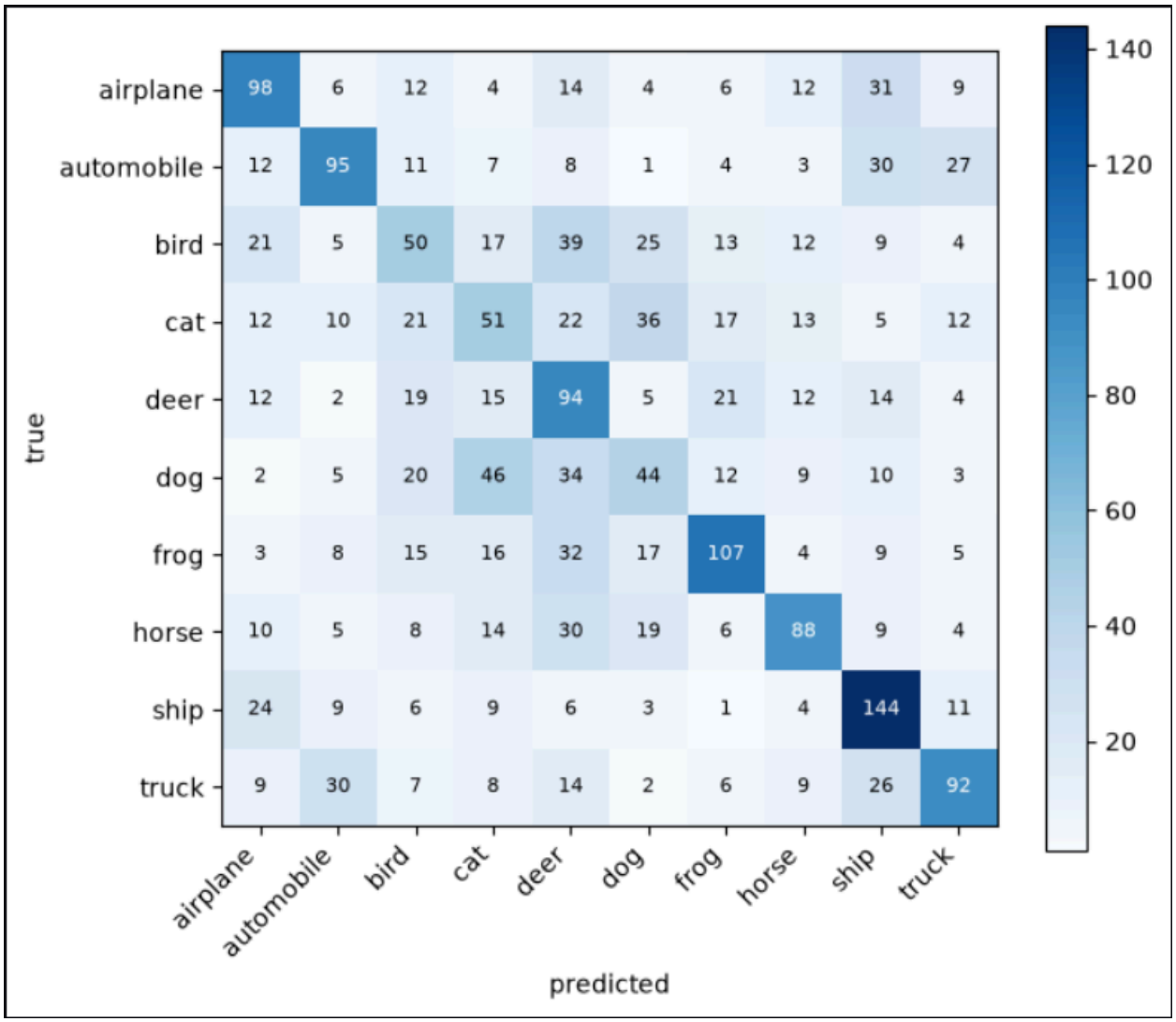


Figure 6. Confusion matrix for the 2,000 CIFAR-10 test images.

Class	Test accuracy
Airplane	50.00%
Automobile	47.98%
Bird	25.64%
Cat	25.63%
Deer	47.47%
Dog	23.78%
Frog	49.54%
Horse	45.60%
Ship	66.36%
Truck	45.32%

Best class Ship (66.36%): Ships often have distinctive backgrounds such as water and sky. These broad color patterns can be easier for an MLP to recognize.

Worst class Dog (23.78%): Dogs can look similar to cats and other animals in color, texture and shape. The model confused 46 of the 185 test dog images with cats. Since the MLP works with flattened pixels, learning detailed local visual features is difficult.

Effect of L2 regularization

We trained the network again using $\lambda = 0.001$.

Metric	No regularization	L2 regularization
Best validation epoch	15	7
Train accuracy at selected epoch	96.99%	78.74%
Validation accuracy at selected epoch	43.30%	43.40%
Train–validation gap	53.69 pp	35.34 pp

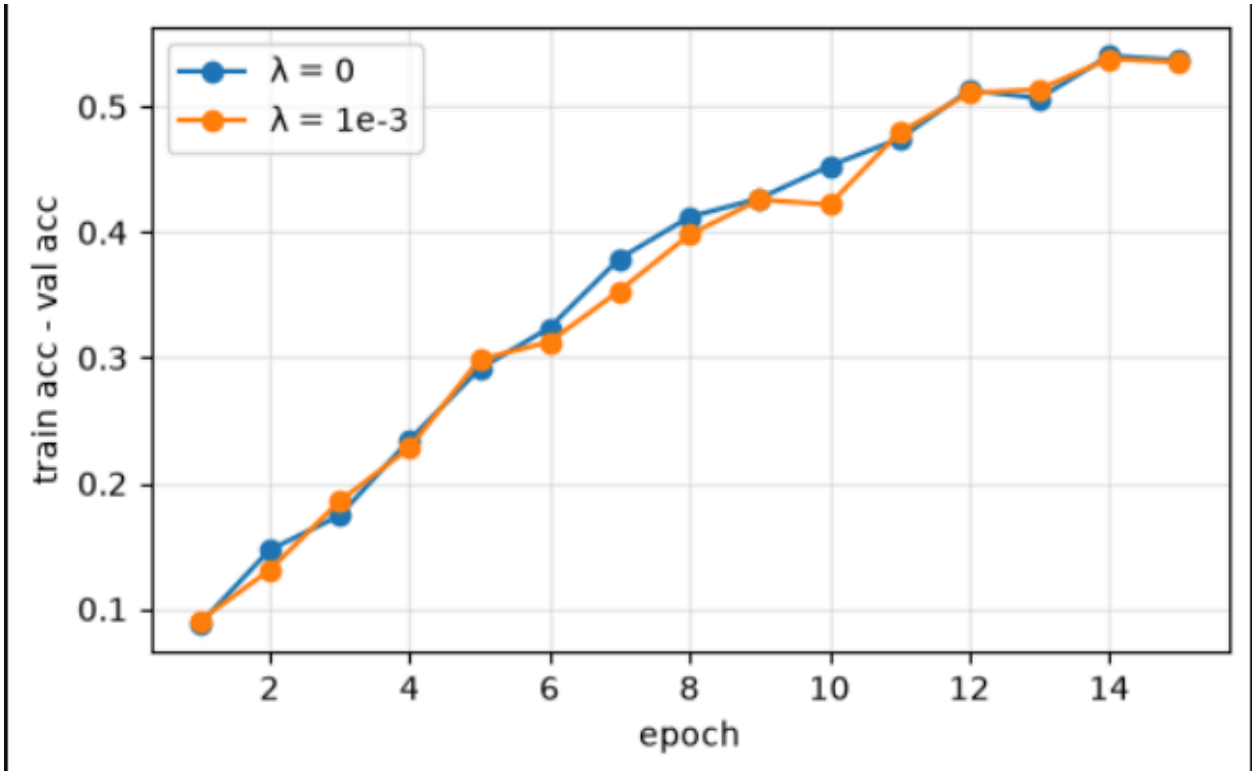


Figure 7. Training and validation performance with L2 regularization.

At the selected best epochs, L2 regularization reduced the train–validation accuracy gap, but the validation accuracy improved by only 0.10 percentage points.

At epoch 15, the gap was still large for both models (53.69 versus 53.53 percentage points). Therefore, the regularization helped limit weight growth but did not fully solve overfitting.

Concept checks

(i) About 95.84% of the parameters are in the first layer. This large number of parameters makes it easy for the network to memorize training images, increasing its tendency to overfit.

(ii) The first-layer weight images show patterns learned directly from pixel values. They can reveal color and spatial preferences, but not necessarily meaningful objects.

(iii) Ideally, training and validation accuracies should be reasonably close while both remain high. A large gap, like the one observed here, indicates poor generalization and overfitting.

8. Gradient Flow in a Deep PyTorch Network (P8)

Experimental setup

We built four deep fully connected networks to study how activation functions and weight initialization affect gradient flow.

Each network had 10 hidden layers of width 128, an input size of 3,072 and 10 output logits. We used a batch of 256 CIFAR-10 training images.

The four configurations were sigmoid with Xavier initialization, sigmoid with He initialization, ReLU with Xavier initialization, and ReLU with He initialization.

Gradient comparison

We measured the mean absolute weight gradient at each linear layer after one forward and backward pass.

Configuration	First-layer gradient	Last-layer gradient
Sigmoid + Xavier	3.035×10^{-9}	1.760×10^{-2}

Sigmoid + He	4.552×10^{-8}	1.867×10^{-2}
ReLU + Xavier	1.842×10^{-4}	4.231×10^{-4}
ReLU + He	6.361×10^{-3}	4.106×10^{-2}

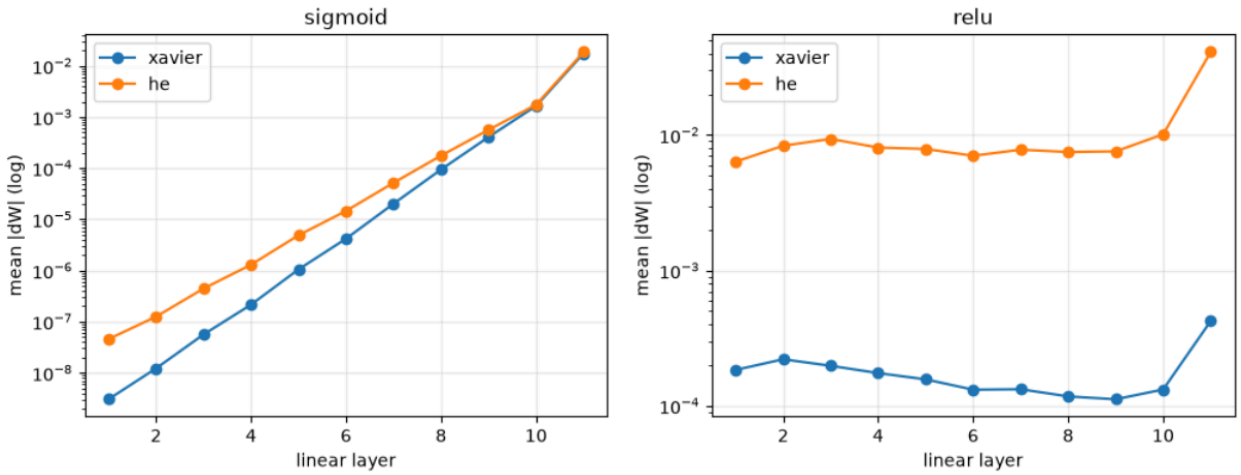


Figure 8. Mean absolute weight gradients across the 11 linear layers for the four network configurations.

The sigmoid networks have extremely small gradients in their early layers, indicating severe vanishing gradients.

ReLU with He initialization has much larger early-layer gradients. This suggests that it should be easier to train than the sigmoid configurations.

Activation standard deviation

We also recorded the standard deviation of the outputs of every linear layer for ReLU networks.

Layer	ReLU + He	ReLU + Xavier
1	1.4185	1.3898

3	1.4497	0.7102
5	1.3686	0.3352
7	1.0364	0.1269
9	1.2044	0.0738
11	1.3011	0.0543

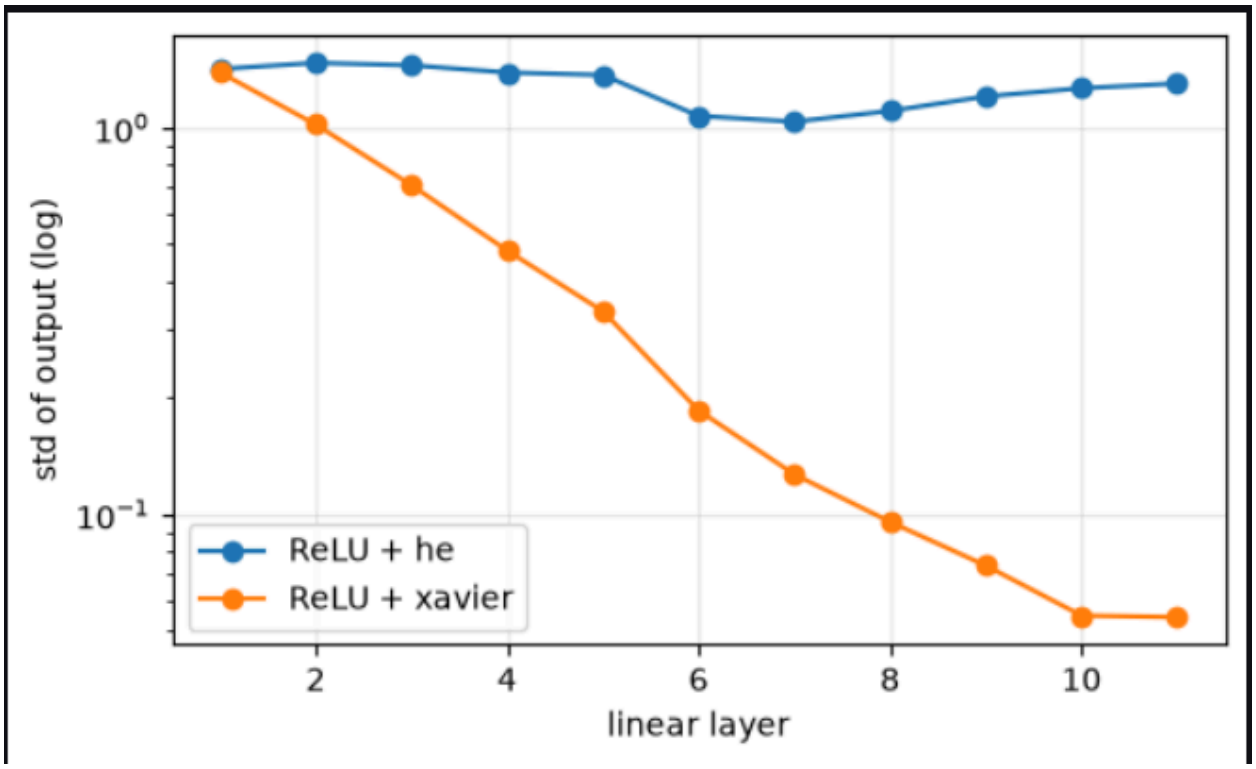


Figure 9. Standard deviation of linear-layer outputs for ReLU networks with different initialization methods.

With He initialization, the output standard deviation stays relatively stable across layers. With Xavier initialization, it decreases substantially as the network becomes deeper.

This supports the idea that He initialization is better suited to ReLU networks because it helps preserve the scale of signals through multiple layers.

Effect of batch normalization

We added batch normalization after each hidden linear layer in the sigmoid network with Xavier initialization.

Configuration	First-layer mean absolute gradient
Without batch normalization	3.035×10^{-9}
With batch normalization	1.466×10^{-3}

The first-layer gradient increased by approximately 4.8×10^{5} times.

This is a major improvement in gradient flow for the tested batch.

Concept checks

(i) We expect ReLU with He initialization to train most reliably because its gradients remain relatively strong across layers. ReLU with Xavier may still train, but signal magnitudes decrease. Both sigmoid networks are likely to train slowly because of vanishing gradients, especially sigmoid with Xavier initialization.

(ii) Batch normalization helps keep the inputs to sigmoid layers at a more suitable scale, reducing saturation. As a result, gradients can pass more effectively through the network, making optimization easier.

9. Autograd Versus Manual Backpropagation (P9 — Bonus)

Experimental setup

We compared our NumPy backpropagation implementation with PyTorch autograd using the same network and parameters.

Network architecture: $13 \rightarrow 5 \rightarrow 4 \rightarrow 3$

We used the first eight standardized Wine training samples, ReLU hidden activations, float64 precision and L2 regularization with $\lambda = 0.01$.

Comparing losses and parameter gradients

Metric	Result
NumPy loss	0.9729251425
PyTorch loss	0.9729251425
Maximum parameter-gradient difference	5.7×10^{-17}
Required maximum difference	1×10^{-9}

The losses matched, and the maximum gradient difference was far below the required threshold.

This confirms that our manual backpropagation agrees with PyTorch autograd for the tested network.

Comparing pre-activation gradients

Gradient	Maximum absolute difference
dZ1	1.7×10^{-17}
dZ2	1.4×10^{-17}

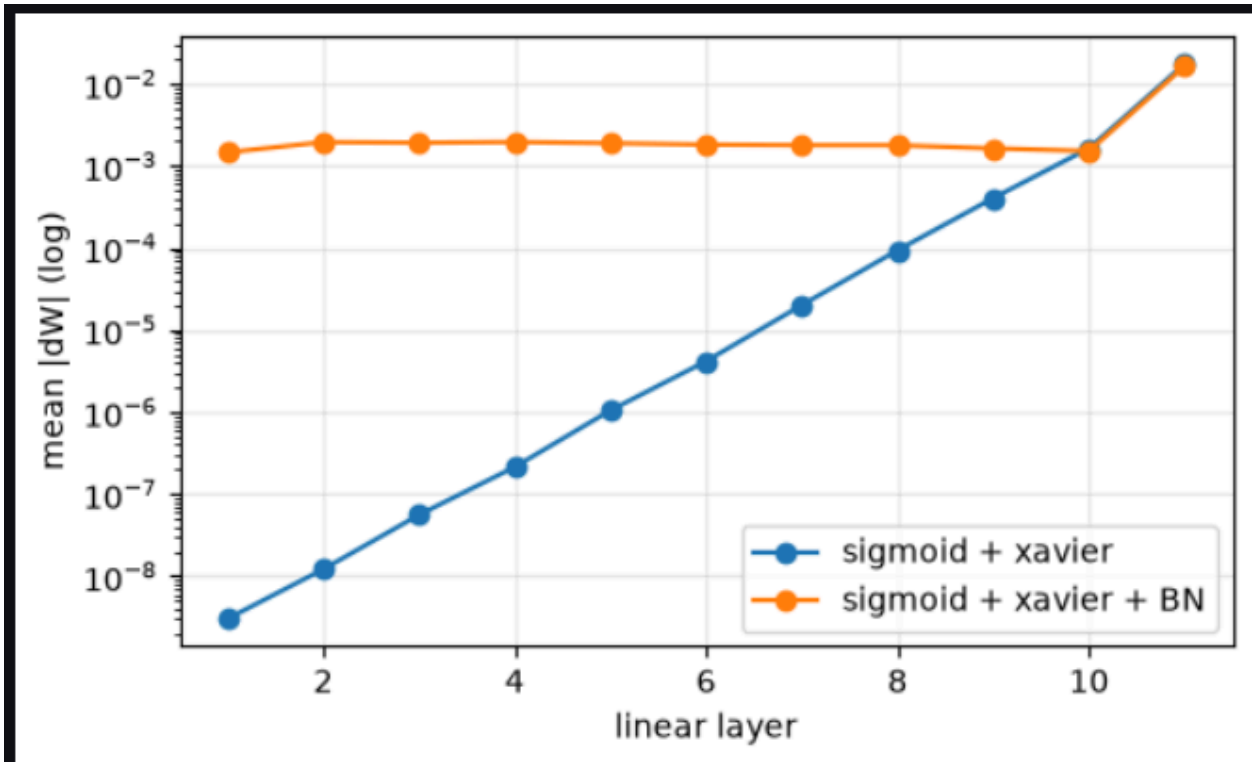
dZ3	1.4×10^{-17}

The gradients at all three layers matched within floating-point precision.

Inspecting the computation graph

We inspected the backward-function nodes connected to the loss tensor.

Backward node	Corresponding operation
MmBackward0	Matrix multiplication
AddBackward0	Addition, including biases
ReluBackward0	ReLU activation
LogSoftmaxBackward0	Log-softmax
NllLossBackward0	Negative log-likelihood
PowBackward0	Squaring weights
SumBackward0	Summing squared weights
MulBackward0	Multiplication by regularization coefficient
AccumulateGrad	Accumulating parameter gradients



Output 10. Backward-function chain showing how PyTorch tracks the operations used to calculate the loss.

The computation graph shows how PyTorch connects forward operations with their corresponding backward operations. This allows autograd to calculate gradients using the chain rule.

Gradient accumulation

We called backward twice on the same computation graph while retaining it.

After the second backward call, every parameter gradient doubled. Resetting the gradients cleared the previously accumulated values.

This demonstrates that PyTorch accumulates gradients by default instead of automatically replacing them.

Stopping gradient tracking

We tested two approaches.

Using `torch.no_grad()`: The forward pass did not create a gradient-tracking graph. The resulting loss had `requires_grad=False` and no `grad_fn`.

Detaching the first hidden activation: The data-loss gradient could no longer propagate to the first layer. However, the first-layer weights still received the L2 penalty gradient, λW_1, because the regularization term directly depends on those weights.

The first-layer bias gradient was `None`, since the bias was not included in the penalty.

Concept checks

(i) PyTorch cross-entropy accepts logits because it combines log-softmax and negative log-likelihood in a numerically stable calculation.

(ii) Gradients accumulate by default so that contributions from multiple backward passes can be combined. We must reset them when starting a new optimization step unless accumulation is intentional.

(iii) We may detach a tensor when we want to stop gradients from flowing through a particular part of a computation, such as when treating intermediate features as fixed or separating parts of a training process.

10. Overall Conclusion

In this assignment, we implemented the main components of neural network training, starting with activation functions and numerical differentiation, then moving to forward propagation, backpropagation and complete MLP training.

Our gradient checks showed that the analytical derivatives were consistent with numerical calculations. We also found that the CIFAR-10 MLP could fit training images well but generalized poorly, showing the limitations of using a large fully connected network on flattened images.

The deep-network experiments demonstrated that activation functions, initialization and batch normalization strongly affect gradient flow. Finally, the bonus experiment confirmed that our NumPy backpropagation matched PyTorch autograd to floating-point precision.

Overall, the experiments helped us understand not only how neural networks are trained, but also why numerical stability, correct gradients and generalization matter.